DR. A. Q. KHAN INSTITUTE OF COMPUTER SCIENCES AND INFORMATION TECHNOLOGY
KAHUTA

LEAFGUARD AI

AI-Based Plant Leaf Disease Detection Application

PRACTICAL REPORT

Student Name:	Maira Masood
Roll Number:	242201061
Degree Program:	BS Computer Science
Class:	BSCS-5B
Course:	Mobile Application Development
Instructor:	Sir Uzair
Submission Date:	04 November 2026

Fall 2026

Declaration

I declare that this practical report describes the development and implementation work for my project, LeafGuard AI. The code, screenshots, and results included in this report should represent the actual work performed for the project.

Acknowledgement

I am thankful to Almighty Allah for giving me the strength to complete this project. I also express my gratitude to my instructor, Sir Uzair, for providing guidance and learning opportunities in Mobile Application Development.

Abstract

LeafGuard AI is an Android-based plant leaf disease detection project that demonstrates the integration of machine learning into a mobile application.

The project uses a TensorFlow Lite model to process an image of a plant leaf and produce a prediction from the model's available disease classes. The purpose is to demonstrate on-device machine learning, where inference can be performed inside an Android application without requiring a server for every prediction.

The practical work includes dataset preparation, model training, TensorFlow Lite conversion, Android integration, and application interface development.

Contents

1 Introduction **ix**

1.1 Project Overview ix

1.2 Problem Statement ix

1.3 Project Objectives ix

1.4 Main Features x

1.5 Tools and Technologies x

2 Dataset Preparation **xi**

2.1 PlantVillage Dataset xi

2.2 Dataset Organization xi

2.3 Image Preprocessing xi

2.4 Training and Validation Data xii

3 Google Colab Model Training **xiii**

3.1 Introduction xiii

3.2 Step 1: Install Required Libraries xiii

3.3 Step 2: Import Libraries xiii

3.4 Step 3: Load PlantVillage Dataset xiii

3.5 Step 4: Split the Dataset xiv

3.6 Step 5: Image Preprocessing xiv

3.7 Step 6: Load MobileNetV2 xv

3.8 Step 7: Build the Classification Model xv

3.9 Step 8: Compile the Model xvi

3.10 Step 9: Train the Model xvi

3.11 Step 10: Evaluate the Model xvi
3.12 Step 11: Save the Trained Model xvi
3.13 Step 12: Convert the Model to TensorFlow Lite xvii
3.14 Step 13: Save Class Labels xvii
3.15 Step 14: Download Model and Labels xvii

4 TensorFlow Lite Model xviii

4.1 What is TensorFlow Lite? xviii
4.2 Model Input and Output xviii
4.3 Model Inference Process xviii
4.4 Important Model Considerations xix

5 Android Application Development xx

5.1 Android Studio xx
5.2 Application Screens xx
5.3 Android Project Structure xx
5.4 Add TensorFlow Lite Dependency xxi
5.5 Place Model Files in Assets xxi
5.6 Load the TensorFlow Lite Model xxi
5.7 Create the Interpreter xxii
5.8 Prediction Workflow xxii
5.9 Application Interface xxii

6 Testing and Results xxiii

6.1 Testing Objectives xxiii
6.2 Testing Checklist xxiii
6.3 Limitations xxiii

7 Project Screenshots xxiv

7.1 Dataset, Training and Implementation Screenshots xxiv
7.2 Final Android Application Screenshots xlv

8 Conclusion l

References li

List of Figures

7.1 Project implementation screenshot 1 xxiv

7.2 Project implementation screenshot 2 xxv

7.3 Project implementation screenshot 3 xxvi

7.4 Project implementation screenshot 4 xxvii

7.5 Project implementation screenshot 5 xxviii

7.6 Project implementation screenshot 6 xxix

7.7 Project implementation screenshot 7 xxx

7.8 Project implementation screenshot 8 xxxi

7.9 Project implementation screenshot 9 xxxii

7.10 Project implementation screenshot 10 xxxiii

7.11 Project implementation screenshot 11 xxxiv

7.12 Project implementation screenshot 12 xxxv

7.13 Project implementation screenshot 13 xxxvi

7.14 Project implementation screenshot 14 xxxvii

7.15 Project implementation screenshot 15 xxxviii

7.16 Project implementation screenshot 16 xxxix

7.17 Project implementation screenshot 17 xl

7.18 Project implementation screenshot 18 xli

7.19 Project implementation screenshot 19 xlii

7.20 Project implementation screenshot 20 xliii

7.21 Project implementation screenshot 21 xliv

7.22 LeafGuard AI Android application screen 22 xlv

7.23 LeafGuard AI Android application screen 23 xlvi

7.24 LeafGuard AI Android application screen 24 xlvii
7.25 LeafGuard AI Android application screen 25 xlviii
7.26 LeafGuard AI Android application screen 26 xlix

1. Introduction

1.1 Project Overview

LeafGuard AI is an AI-based Android application designed to demonstrate plant leaf disease classification.

The user selects a leaf image from the device. The image is prepared for the machine learning model. The TensorFlow Lite model processes the image and returns prediction scores for its supported classes.

1.2 Problem Statement

Plant diseases can affect crop quality and production. Identifying possible diseases from leaf images can be difficult for beginners.

This project demonstrates how artificial intelligence can be used in a mobile application to classify leaf images.

1.3 Project Objectives

- To understand machine learning in Android.
- To prepare a plant leaf image dataset.
- To train an image classification model.
- To convert the trained model into TensorFlow Lite.
- To integrate the model into an Android application.
- To display the model's prediction in the application.

1.4 Main Features

- Splash screen.
- Login interface.
- Home or dashboard screen.
- Leaf image selection interface.
- AI-based image classification workflow.
- Prediction result interface.

1.5 Tools and Technologies

Tool	**Purpose**
Android Studio	Android application development
Java	Android application programming
XML	Designing application interfaces
Python	Model training and preparation
Google Colab	Cloud-based model training
TensorFlow	Machine learning framework
TensorFlow Lite	Running the model on Android
PlantVillage	Plant leaf image dataset
GitHub	Source code repository

2. Dataset Preparation

2.1 PlantVillage Dataset

PlantVillage is a publicly available dataset containing images of plant leaves. It includes different plant categories and disease classes.

The classification task uses images organized into separate classes. Each class represents a particular plant and leaf condition.

2.2 Dataset Organization

For directory-based training, the dataset should have one folder for each class.

Example structure:

```
PlantVillage/
    Apple___Apple_scab/
    Apple___Black_rot/
    Apple___healthy/
    Tomato___Early_blight/
    Tomato___Late_blight/
    Tomato___healthy/
    ...
```

The complete dataset contains multiple class folders. The folder names are used as class labels by the training program.

2.3 Image Preprocessing

Before training, images are resized to 224 by 224 pixels.

The images are converted into numerical values so that the neural network can process them.

MobileNetV2 preprocessing scales image values into the range expected by the model.

2.4 Training and Validation Data

The dataset is divided into two parts:

- Training data: used to teach the model.
- Validation data: used to evaluate its performance on images not used for training.

3. Google Colab Model Training

3.1 Introduction

Google Colab provides a cloud-based Python environment. It can be used to prepare the dataset, train the classification model, evaluate the model, and export the model into TensorFlow Lite format.

3.2 Step 1: Install Required Libraries

Run this cell in Google Colab.

```
!pip install -q tensorflow tensorflow-datasets
!pip install -q importlib-resources
```

3.3 Step 2: Import Libraries

```
import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
import json
import os

print("TensorFlow:", tf.__version__)
print("TensorFlow Datasets:", tfds.__version__)
```

3.4 Step 3: Load PlantVillage Dataset

The following code loads the PlantVillage dataset through TensorFlow Datasets.

```
dataset, info = tfds.load(
    "plant_village",
    split="train",
    as_supervised=True,
    with_info=True
)

```

```
class_names = info.features["label"].names
NUM_CLASSES = len(class_names)

print("Number of classes:", NUM_CLASSES)
print("Class names:")
print(class_names)
```

3.5 Step 4: Split the Dataset

The dataset is divided into training and validation sets. A fixed random seed makes the split repeatable.

```
dataset_size = info.splits["train"].num_examples

train_size = int(dataset_size * 0.8)

shuffled = dataset.shuffle(
    dataset_size,
    seed=42,
    reshuffle_each_iteration=False
)

train_data = shuffled.take(train_size)
val_data = shuffled.skip(train_size)

print("Training examples:", train_size)
print("Validation examples:",
      dataset_size - train_size)
```

3.6 Step 5: Image Preprocessing

Images are resized and preprocessed for MobileNetV2.

```
IMG_SIZE = 224
BATCH_SIZE = 32

def preprocess_image(image, label):
    image = tf.image.resize(
        image,
        (IMG_SIZE, IMG_SIZE)
    )

    image = tf.cast(image, tf.float32)

    image = tf.keras.applications.mobilenet_v2.preprocess_input(
        image
    )

    return image, label

train_data = (
    train_data
```

```
    .map(preprocess_image,
         num_parallel_calls=tf.data.AUTOTUNE)
    .shuffle(1000)
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)

val_data = (
    val_data
    .map(preprocess_image,
         num_parallel_calls=tf.data.AUTOTUNE)
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)
```

3.7 Step 6: Load MobileNetV2

MobileNetV2 is a lightweight convolutional neural network that can be used for image classification.

Transfer learning uses an already trained model as the starting point for a new classification task.

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False
```

3.8 Step 7: Build the Classification Model

A Global Average Pooling layer and a final Dense layer are added to classify the plant leaf images.

```
inputs = tf.keras.Input(
    shape=(224, 224, 3)
)

x = base_model(
    inputs,
    training=False
)

x = tf.keras.layers.GlobalAveragePooling2D()(x)

x = tf.keras.layers.Dropout(0.2)(x)

outputs = tf.keras.layers.Dense(
    NUM_CLASSES,
```

```
    activation="softmax"
)(x)

model = tf.keras.Model(
    inputs=inputs,
    outputs=outputs
)

model.summary()
```

3.9 Step 8: Compile the Model

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

3.10 Step 9: Train the Model

The model learns patterns from the training images.

```
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

The number of epochs can be changed according to available computing resources and training requirements.

3.11 Step 10: Evaluate the Model

```
loss, accuracy = model.evaluate(val_data)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
```

The actual values printed by Google Colab should be recorded in the final report. No accuracy value should be assumed without running the training and evaluation.

3.12 Step 11: Save the Trained Model

```
model.save("leafguard_model.keras")

print("Model saved successfully.")
```

3.13 Step 12: Convert the Model to TensorFlow Lite

TensorFlow Lite conversion creates a model file that can be used for on-device inference.

```
converter = tf.lite.TFLiteConverter.from_keras_model(
    model
)

tflite_model = converter.convert()

with open("leaf_model.tflite", "wb") as f:
    f.write(tflite_model)

print("TensorFlow Lite model exported.")
```

3.14 Step 13: Save Class Labels

The labels must remain in exactly the same order as the model's output classes.

```
with open("labels.txt", "w") as f:
    for name in class_names:
        f.write(name + "\n")

print("Labels saved.")
```

3.15 Step 14: Download Model and Labels

```
from google.colab import files

files.download("leaf_model.tflite")
files.download("labels.txt")
```

Important: The model input preprocessing and label order used in Android must exactly match the training setup. The code above creates a new model; it does not prove that an already existing model was trained using this exact code.

4. TensorFlow Lite Model

4.1 What is TensorFlow Lite?

TensorFlow Lite is a framework for running machine learning models on mobile and embedded devices.

It allows Android applications to perform model inference locally on the device.

4.2 Model Input and Output

The inspected model information for the project is:

Property	**Value**
Input tensor shape	[1, 224, 224, 3]
Input data type	float32
Output tensor shape	[1, 38]
Output data type	float32
Number of output classes	38

These details describe the inspected model. The training code, model file, and Android preprocessing must be consistent before using the model for predictions.

4.3 Model Inference Process

The inference process follows these steps:

1. User selects a leaf image.
2. The image is decoded.
3. The image is resized to 224 by 224 pixels.
4. Pixel values are preprocessed.

5. The image is passed to the TensorFlow Lite model.
6. The model produces 38 output scores.
7. The application identifies the highest score.
8. The corresponding class label is displayed.

4.4 Important Model Considerations

The Android application must use the same image normalization and class-label order as the trained model.

A prediction score should only be displayed as a percentage confidence when the output has been verified to be a probability distribution, such as softmax output.

5. Android Application Development

5.1 Android Studio

Android Studio is the official integrated development environment used for Android application development.

It provides tools for writing code, designing interfaces, building applications, and testing them.

5.2 Application Screens

The LeafGuard AI application includes the following interface screens:

- Splash screen.
- Login screen.
- Home screen.
- Leaf image selection screen.
- Detection result screen.

5.3 Android Project Structure

An example project structure is shown below.

```
app/
  src/
    main/
      java/
        com/example/leafguardai/
          MainActivity.java
      assets/
        leaf_model.tflite
        labels.txt
      res/
```

```
        layout/
        drawable/
        mipmap/
      AndroidManifest.xml
```

5.4 Add TensorFlow Lite Dependency

The TensorFlow Lite dependency is added to the application-level Gradle file.

```
dependencies {
    implementation("org.tensorflow:tensorflow-lite:2.17.0")
}
```

The dependency version should be checked against the Android project's Gradle and TensorFlow Lite compatibility.

5.5 Place Model Files in Assets

The following files should be placed inside the assets folder:

- leaf$_m odel.tflite$

- labels.txt

The model and labels must belong to the same trained model version.

5.6 Load the TensorFlow Lite Model

The Interpreter loads the model file and runs inference.

```
import org.tensorflow.lite.Interpreter;
import android.content.res.AssetFileDescriptor;
import java.io.FileInputStream;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;

private MappedByteBuffer loadModel()
        throws Exception {

    AssetFileDescriptor fileDescriptor =
        getAssets().openFd("leaf_model.tflite");

    FileInputStream inputStream =
        new FileInputStream(
            fileDescriptor.getFileDescriptor()
```

```
        );

    FileChannel fileChannel =
        inputStream.getChannel();

    return fileChannel.map(
        FileChannel.MapMode.READ_ONLY,
        fileDescriptor.getStartOffset(),
        fileDescriptor.getDeclaredLength()
    );
}
```

5.7 Create the Interpreter

```
Interpreter tflite;

try {
    tflite = new Interpreter(loadModel());
} catch (Exception e) {
    e.printStackTrace();
}
```

5.8 Prediction Workflow

The application prepares the selected image and passes a correctly shaped input tensor to the model.

For the inspected model, the expected input shape is [1, 224, 224, 3], and the output shape is [1, 38].

The exact preprocessing must match the model's training preprocessing. The output index must also match the same index in labels.txt.

5.9 Application Interface

The application interface is designed to provide a simple workflow for selecting a leaf image and viewing the classification result.

The login interface in a demonstration project should not be considered secure authentication unless a proper authentication system has been implemented.

6. Testing and Results

6.1 Testing Objectives

Testing is necessary to check whether the application screens, image selection workflow, model loading, and prediction process work correctly.

6.2 Testing Checklist

Test Case	**Result**
Application launches	To be verified
Splash screen appears	To be verified
Login screen appears	To be verified
Home screen appears	To be verified
Leaf image can be selected	To be verified
TensorFlow Lite model loads	To be verified
Prediction is displayed	To be verified
Correct class label is displayed	To be verified

Actual results should be entered after performing each test on the Android application.

6.3 Limitations

- Predictions depend on the quality of the input image.
- The model can only classify its supported classes.
- Incorrect preprocessing can produce incorrect results.
- Dataset images may not represent every real-world condition.
- A model prediction is not a replacement for expert diagnosis.

7. Project Screenshots

7.1 Dataset, Training and Implementation Screenshots

The following images provide visual evidence of the dataset preparation, training activities, model development, and implementation process.

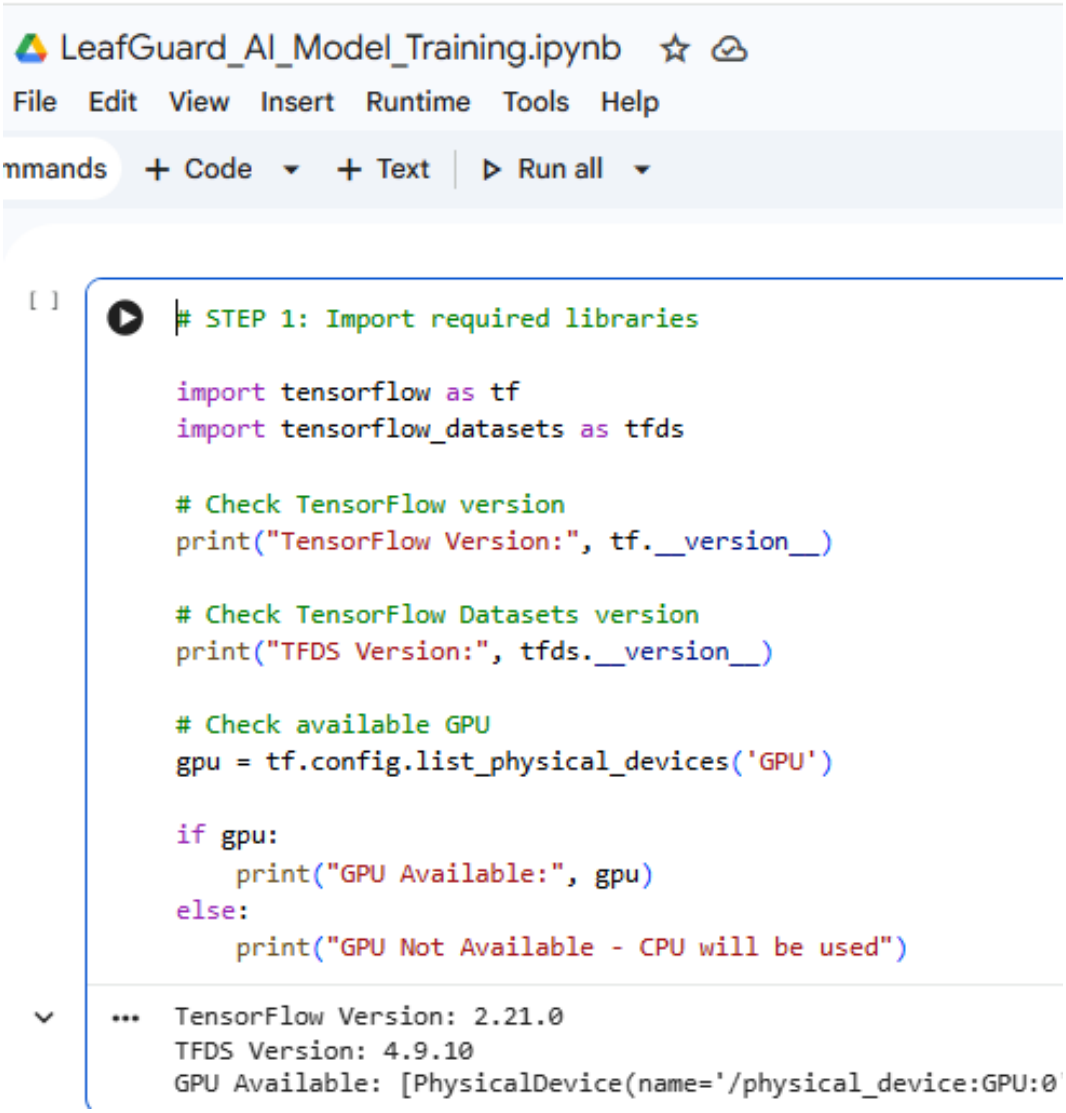


Figure 7.1: Project implementation screenshot 1

```
# STEP 2: Install TensorFlow Datasets

!pip install -q tensorflow-datasets

print("TensorFlow Datasets installed successfully!")
```

```
TensorFlow Datasets installed successfully!
```

```
# STEP 3: Load PlantVillage Dataset

import tensorflow as tf
import tensorflow_datasets as tfds

# Load training, validation and testing datasets
result = tfds.load(
    'plant_village',
    split=[
        'train[:80%]',
        'train[80%:90%]',
        'train[90%:]'
    ],
    as_supervised=True,
    with_info=True
)
```

Figure 7.2: Project implementation screenshot 2

```
# FIX: Install missing dependency

!pip install -q importlib-resources

import importlib_resources

print("Missing package installed successfully!")
```

```
Missing package installed successfully!
```

```
import tensorflow as tf
import tensorflow_datasets as tfds

print("TensorFlow:", tf.__version__)
print("TFDS:", tfds.__version__)

builder = tfds.builder('plant_village')

print("Preparing PlantVillage dataset...")

builder.download_and_prepare()

print("SUCCESS! PlantVillage downloaded.")
```

```
TensorFlow: 2.21.0
TFDS: 4.9.10
Preparing PlantVillage dataset...
Downloading and preparing dataset Unknown size (download: U
```

Figure 7.3: Project implementation screenshot 3

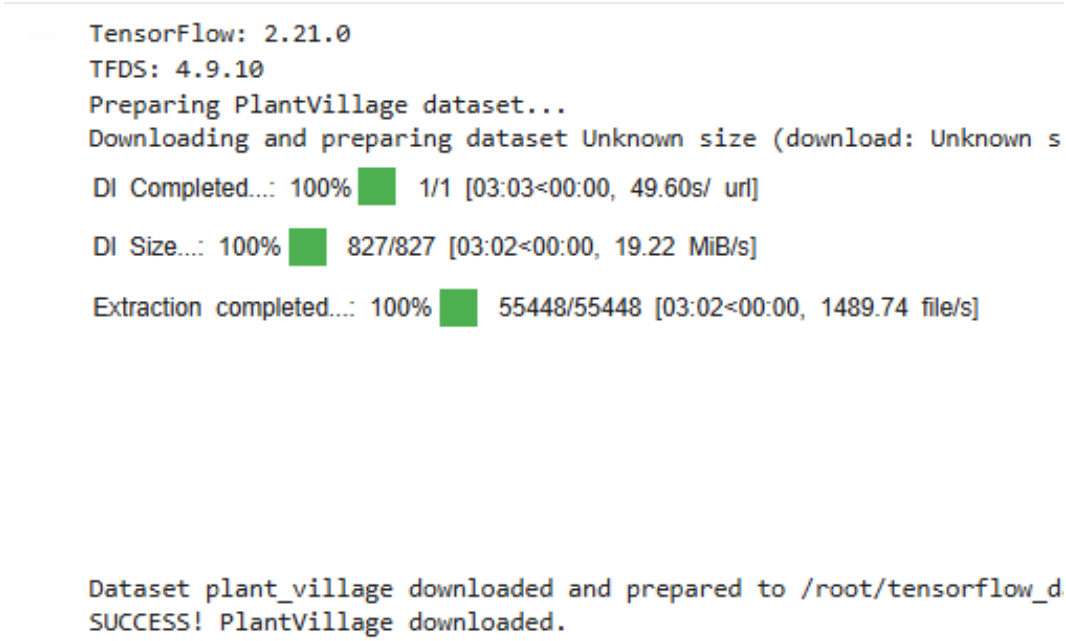


Figure 7.4: Project implementation screenshot 4

```python
import tensorflow as tf
import tensorflow_datasets as tfds

# Load prepared PlantVillage dataset
(train_data, validation_data, test_data), info = tfds.load(
    'plant_village',
    split=[
        'train[:80%]',
        'train[80%:90%]',
        'train[90%:]'
    ],
    as_supervised=True,
    with_info=True
)

print("Dataset loaded successfully!")
print("Total classes:", info.features['label'].num_classes)

print("Training examples:",
      tf.data.experimental.cardinality(train_data).numpy())

print("Validation examples:",
      tf.data.experimental.cardinality(validation_data).numpy())

print("Testing examples:",
      tf.data.experimental.cardinality(test_data).numpy())
```

```
Dataset loaded successfully!
```

Figure 7.5: Project implementation screenshot 5

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))

for images, labels in train_data.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)

        plt.imshow(images.numpy())
        class_name = info.features['label'].names[labels.numpy()]

        plt.title(class_name, fontsize=8)
        plt.axis('off')

plt.tight_layout()
plt.show()
```

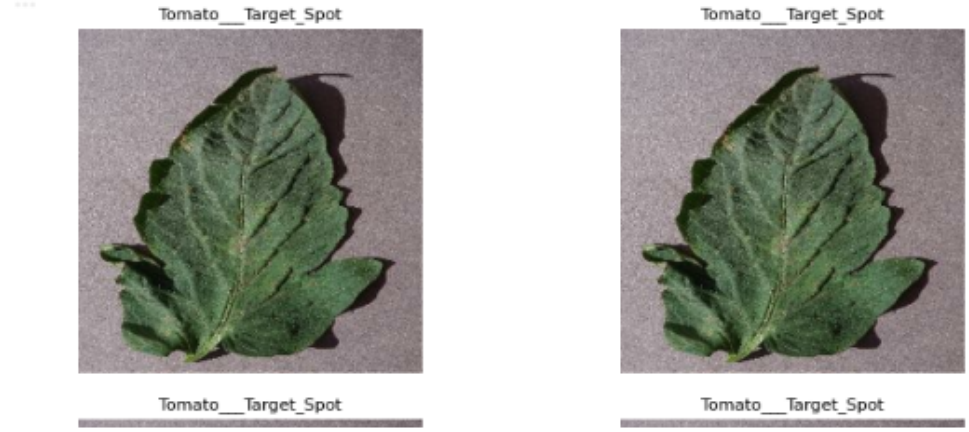


Figure 7.6: Project implementation screenshot 6



Figure 7.7: Project implementation screenshot 7

```
# STEP 5: Define image preprocessing settings

IMG_SIZE = 224
BATCH_SIZE = 32

print("Image Size:", IMG_SIZE, "x", IMG_SIZE)
print("Batch Size:", BATCH_SIZE)
print("Preprocessing settings ready!")
```

```
Image Size: 224 x 224
Batch Size: 32
Preprocessing settings ready!
```

```
# STEP 6: Image preprocessing function

def preprocess_image(image, label):

    # Resize image to 224 x 224 pixels
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))

    # Convert image to float values
    image = tf.cast(image, tf.float32)

    # Prepare pixel values for MobileNetV2
    image = tf.keras.applications.mobilenet_v2.preprocess_input(image)

    return image, label


print("Preprocessing function created!")
```

```
Preprocessing function created!
```

Figure 7.8: Project implementation screenshot 8

```
# STEP 7: Prepare training, validation and testing data

AUTOTUNE = tf.data.AUTOTUNE

# Apply preprocessing
train_data = train_data.map(
    preprocess_image,
    num_parallel_calls=AUTOTUNE
)

validation_data = validation_data.map(
    preprocess_image,
    num_parallel_calls=AUTOTUNE
)

test_data = test_data.map(
    preprocess_image,
    num_parallel_calls=AUTOTUNE
)

# Shuffle training images
train_data = train_data.shuffle(1000)

# Create batches
train_data = train_data.batch(BATCH_SIZE)
validation_data = validation_data.batch(BATCH_SIZE)
test_data = test_data.batch(BATCH_SIZE)

# Improve data loading performance
train_data = train_data.prefetch(AUTOTUNE)
validation_data = validation_data.prefetch(AUTOTUNE)
```

Figure 7.9: Project implementation screenshot 9

```python
# STEP 8: Check image shape and labels

for images, labels in train_data.take(1):

    print("Image batch shape:", images.shape)
    print("Label batch shape:", labels.shape)
    print("Minimum pixel value:", tf.reduce_min(images).numpy())
    print("Maximum pixel value:", tf.reduce_max(images).numpy())
```

```
Image batch shape: (32, 224, 224, 3)
Label batch shape: (32,)
Minimum pixel value: -1.0
Maximum pixel value: 0.9831933
```

```python
# STEP 9: Import required libraries

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import (
    Dense,
    GlobalAveragePooling2D,
    Dropout
)
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Figure 7.10: Project implementation screenshot 10

```python
# STEP 10: Load MobileNetV2

IMG_SIZE = 224

# Get the number of leaf disease categories
NUM_CLASSES = info.features['label'].num_classes

# Load pretrained MobileNetV2
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

# Freeze pretrained layers
base_model.trainable = False

print("MobileNetV2 loaded successfully!")
print("Number of classes:", NUM_CLASSES)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
MobileNetV2 loaded successfully!
Number of classes: 38
```

Figure 7.11: Project implementation screenshot 11

```
IMG_SIZE = 224

# Get the number of leaf disease categories
NUM_CLASSES = info.features['label'].num_classes

# Load pretrained MobileNetV2
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

# Freeze pretrained layers
base_model.trainable = False

print("MobileNetV2 loaded successfully!")
print("Number of classes:", NUM_CLASSES)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/k
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
MobileNetV2 loaded successfully!
Number of classes: 38
```

Figure 7.12: Project implementation screenshot 12

```python
# STEP 11: Build LeafGuard AI

# Create input layer
inputs = tf.keras.Input(shape=(224, 224, 3))

# Extract image features
x = base_model(inputs, training=False)

# Convert feature maps into a vector
x = GlobalAveragePooling2D()(x)

# Reduce overfitting
x = Dropout(0.2)(x)

# Predict all leaf disease categories
outputs = Dense(
    NUM_CLASSES,
    activation='softmax'
)(x)

# Create final model
model = Model(inputs, outputs)

print("LeafGuard AI model created!")
```

```
LeafGuard AI model created!
```

Figure 7.13: Project implementation screenshot 13

```python
# STEP 12: Compile AI model

model.compile(
    optimizer=Adam(learning_rate=0.001),

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']
)

print("Model compiled successfully!")

# Display model architecture
model.summary()
```

```
Model compiled successfully!
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 38)	48,678

```
Total params: 2,306,662 (8.80 MB)
Trainable params: 48,678 (190.15 KB)
Non-trainable params: 2,257,984 (8.61 MB)
```

Figure 7.14: Project implementation screenshot 14

```python
# STEP 13: Train LeafGuard AI

EPOCHS = 5

history = model.fit(
    train_data,
    validation_data=validation_data,
    epochs=EPOCHS
)

print("LeafGuard AI training completed!")
```

```
Epoch 1/5
1358/1358 ━━━━━━━━━━━━━━━━━━━━ 84s 53ms/step - accuracy: 0.8770 - loss: 0.4485
Epoch 2/5
1358/1358 ━━━━━━━━━━━━━━━━━━━━ 61s 44ms/step - accuracy: 0.9451 - loss: 0.1790
Epoch 3/5
1358/1358 ━━━━━━━━━━━━━━━━━━━━ 60s 43ms/step - accuracy: 0.9544 - loss: 0.1454
Epoch 4/5
1358/1358 ━━━━━━━━━━━━━━━━━━━━ 58s 42ms/step - accuracy: 0.9601 - loss: 0.1239
```

Figure 7.15: Project implementation screenshot 15

```python
# STEP 14: Display training accuracy

import matplotlib.pyplot as plt

plt.figure(figsize=(9, 5))

plt.plot(history.history['accuracy'],
         label='Training Accuracy')

plt.plot(history.history['val_accuracy'],
         label='Validation Accuracy')

plt.title('LeafGuard AI - Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

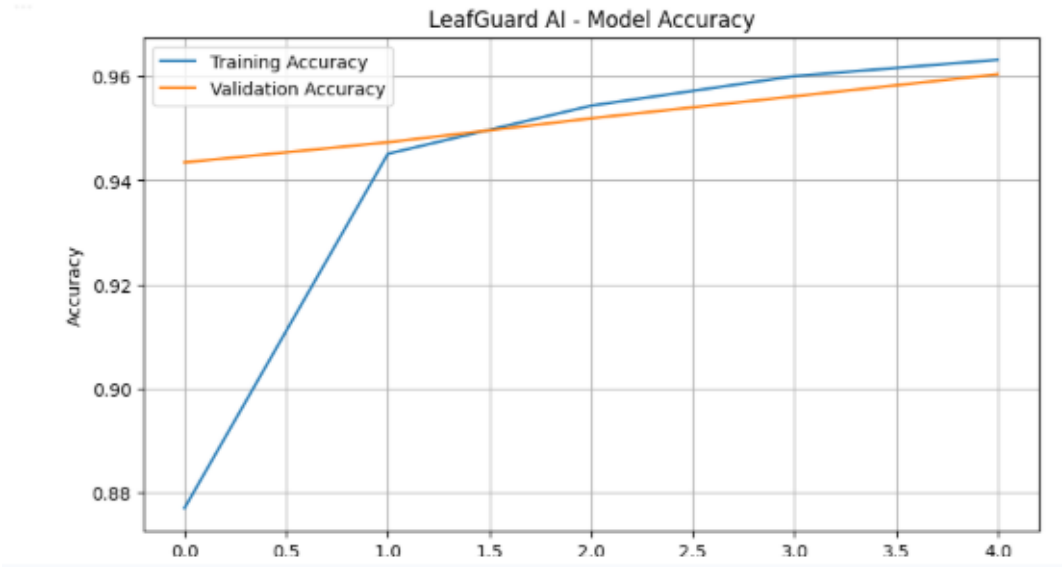


Figure 7.16: Project implementation screenshot 16

```python
# STEP 15: Display training loss

plt.figure(figsize=(9, 5))

plt.plot(history.history['loss'],
         label='Training Loss')

plt.plot(history.history['val_loss'],
         label='Validation Loss')

plt.title('LeafGuard AI - Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```

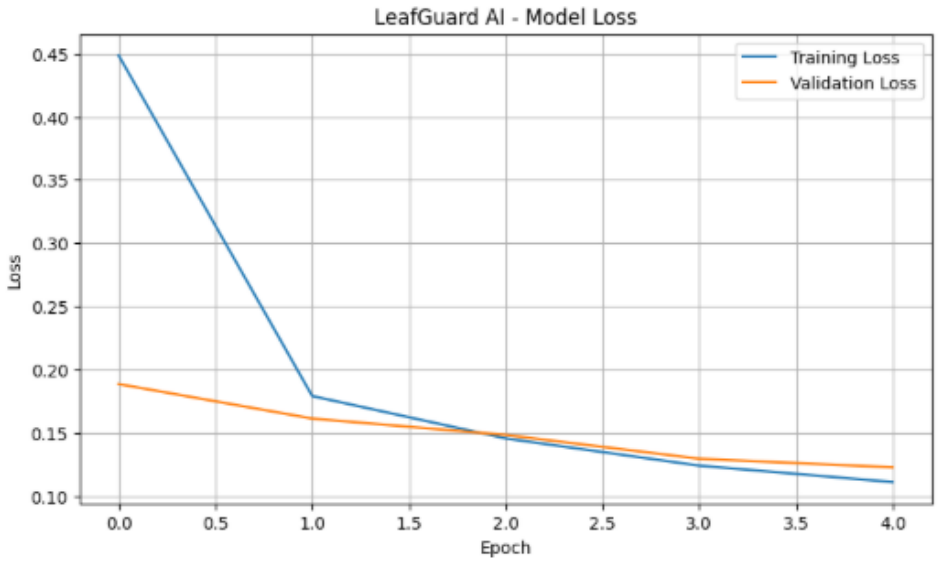


Figure 7.17: Project implementation screenshot 17

```python
# STEP 18: Display leaf image and prediction

plt.figure(figsize=(6, 6))

plt.imshow(image.numpy())
plt.axis('off')

plt.title(
    f"Predicted: {class_names[predicted_index]}\n"
    f"Confidence: {confidence:.2f}%"
)

plt.show()
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow wit
```

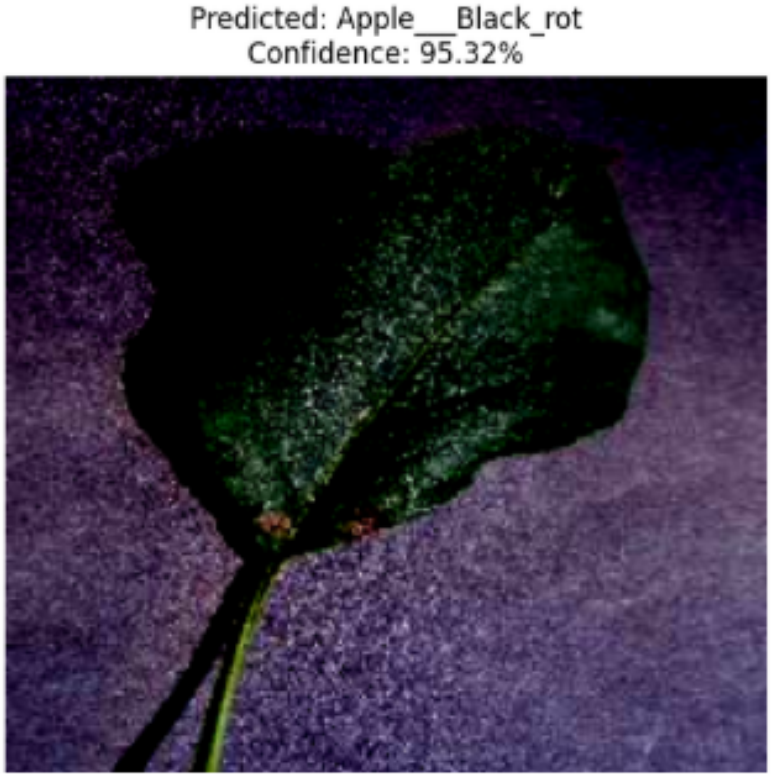


Figure 7.18: Project implementation screenshot 18

```
# STEP 19: Save LeafGuard AI model

model.save("LeafGuard_AI.keras")

print("Model saved successfully!")
```

```
Model saved successfully!
```

```
# STEP 20: Convert Keras model to TFLite

converter = tf.lite.TFLiteConverter.from_keras_model(model)

# Convert model
tflite_model = converter.convert()

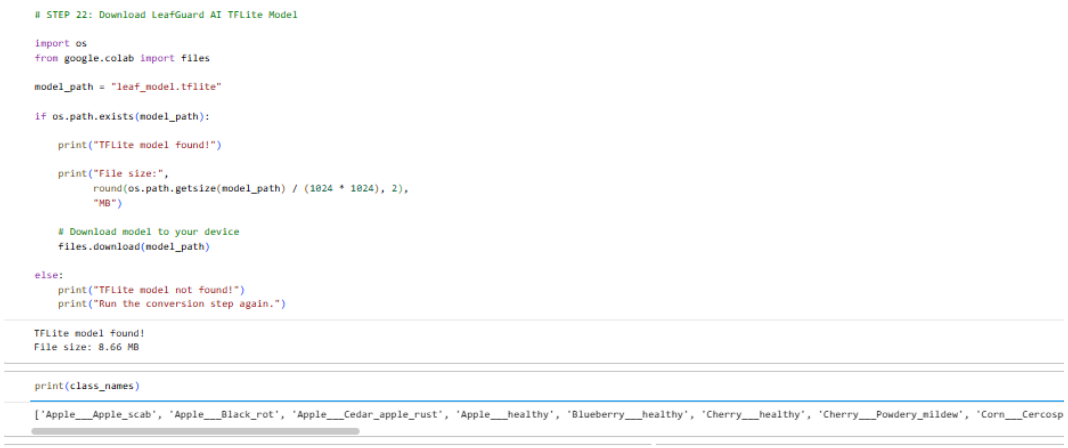
# Save TFLite model
with open("leaf_model.tflite", "wb") as file:
    file.write(tflite_model)

print("TensorFlow Lite conversion successful!")
print("Model size:",
      round(len(tflite_model) / (1024 * 1024), 2),
      "MB")
```

```
Saved artifact at '/tmp/tmp8rms7ez5'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.flo
Output Type:
  TensorSpec(shape=(None, 38), dtype=tf.float32, name=None)
Captures:
  139441405072720: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892959376: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892953424: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892960528: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892959568: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892959952: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892960144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  139440892961104: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

Figure 7.19: Project implementation screenshot 19



```
# STEP 22: Download LeafGuard AI TFLite Model

import os
from google.colab import files

model_path = "leaf_model.tflite"

if os.path.exists(model_path):

    print("TFLite model found!")

    print("File size:",
          round(os.path.getsize(model_path) / (1024 * 1024), 2),
          "MB")

    # Download model to your device
    files.download(model_path)

else:
    print("TFLite model not found!")
    print("Run the conversion step again.")
```

```
TFLite model found!
File size: 8.66 MB
```

```
print(class_names)
```

```
['Apple___Apple_scab', 'Apple___Black_rot', 'Apple___Cedar_apple_rust', 'Apple___healthy', 'Blueberry___healthy', 'Cherry___healthy', 'Cherry___Powdery_mildew', 'Corn___Cercosp
```

Figure 7.20: Project implementation screenshot 20

```
with open("labels.txt", "w") as f:
    for name in class_names:
        f.write(name + "\n")

from google.colab import files
files.download("labels.txt")
```

```
import tensorflow as tf

interpreter = tf.lite.Interpreter(
    model_path="leaf_model.tflite"
)

interpreter.allocate_tensors()

print("INPUT DETAILS:")
print(interpreter.get_input_details())

print("\nOUTPUT DETAILS:")
print(interpreter.get_output_details())
```

```
INPUT DETAILS:
[{'name': 'serving_default_keras_tensor_154:0', 'index': 0, 'shape': array([  1, 224,

OUTPUT DETAILS:
[{'name': 'StatefulPartitionedCall_1:0', 'index': 172, 'shape': array([ 1, 38], dtype=
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: Use
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

Figure 7.21: Project implementation screenshot 21

7.2 Final Android Application Screenshots

This section presents the final application screenshots from LeafGuard AI.

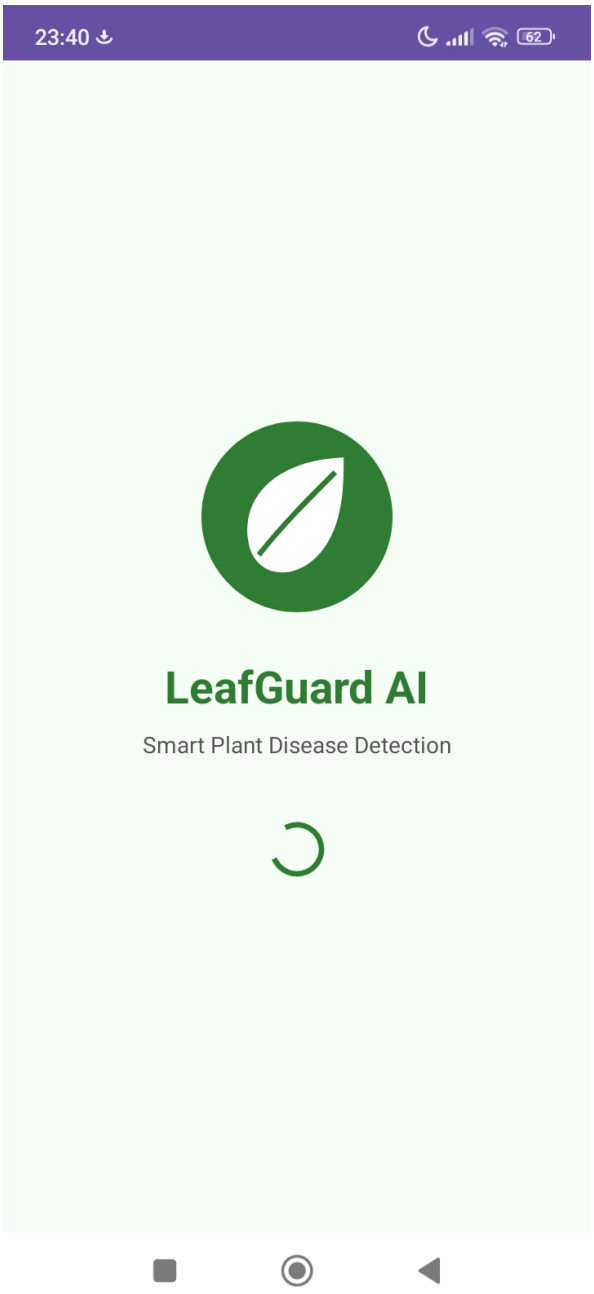


Figure 7.22: LeafGuard AI Android application screen 22

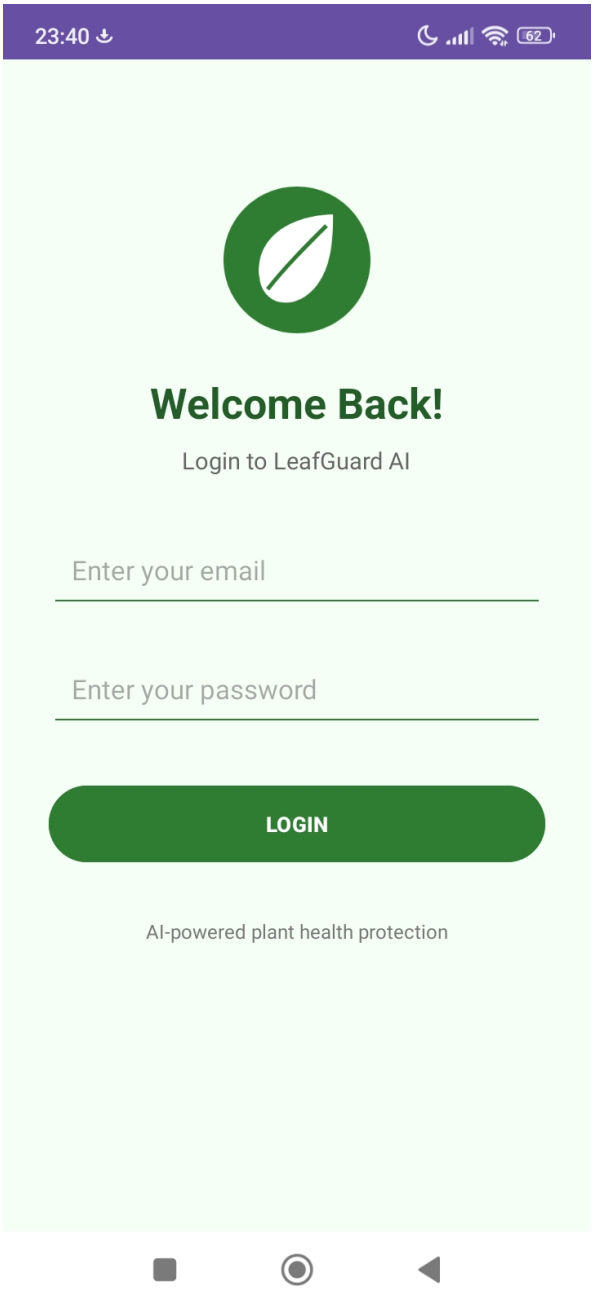


Figure 7.23: LeafGuard AI Android application screen 23

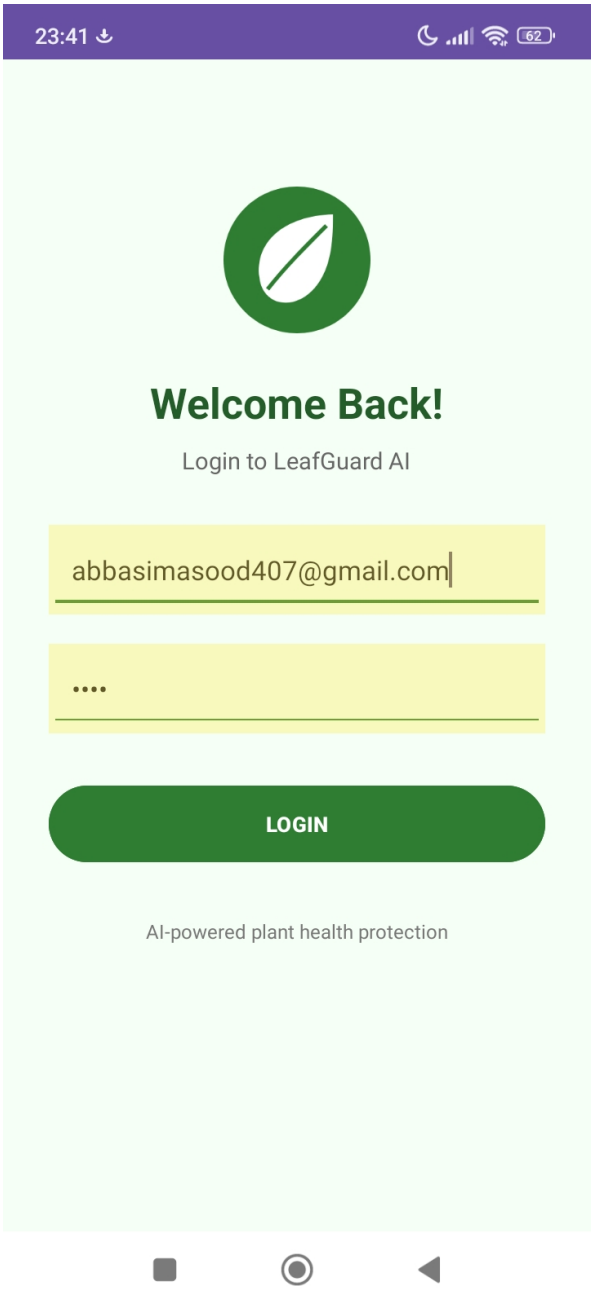


Figure 7.24: LeafGuard AI Android application screen 24

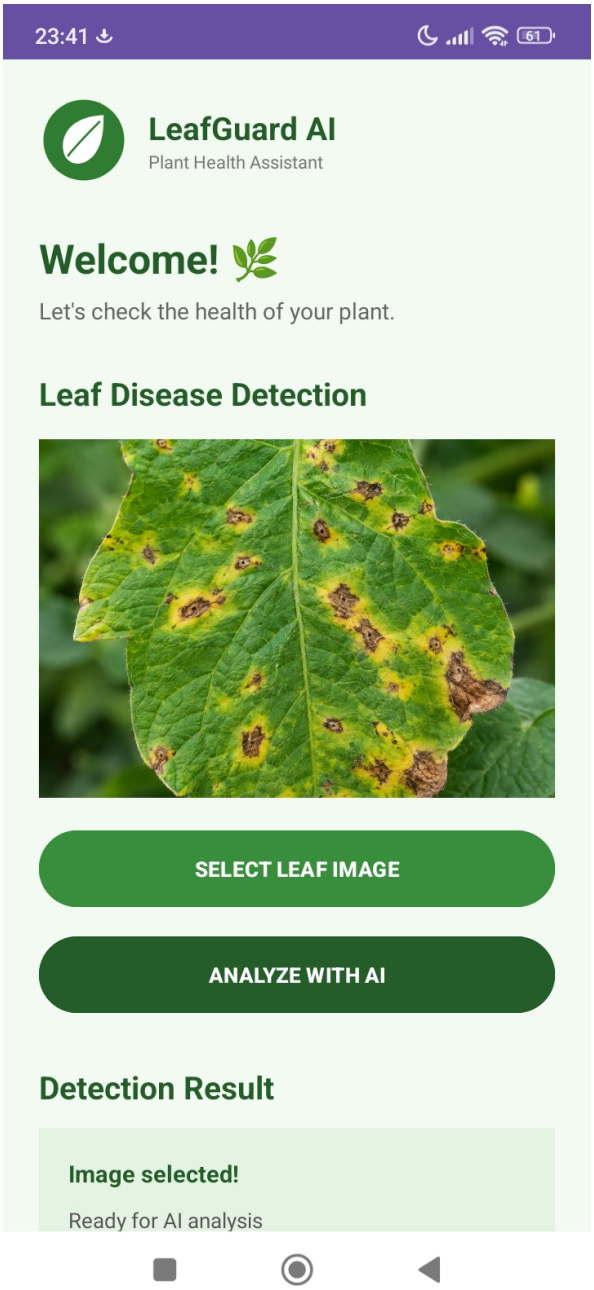


Figure 7.25: LeafGuard AI Android application screen 25

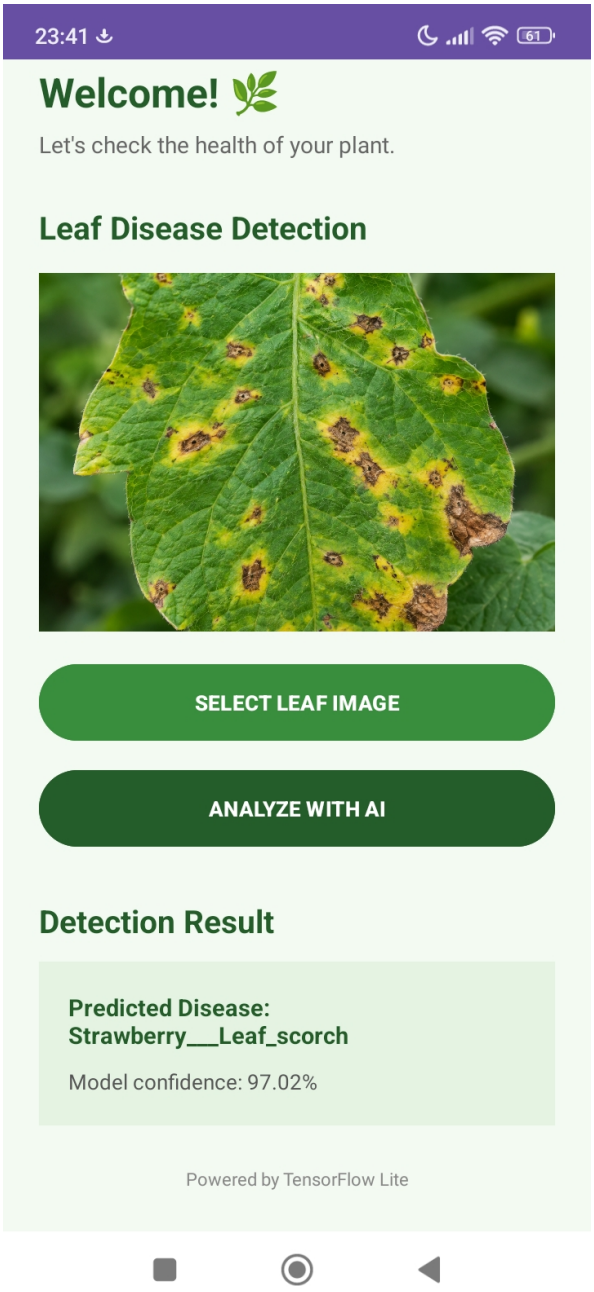


Figure 7.26: LeafGuard AI Android application screen 26

8. Conclusion

LeafGuard AI demonstrates the process of integrating machine learning into an Android application using TensorFlow Lite.

The project covers dataset preparation, image preprocessing, model training, model conversion, Android development, and the application inference workflow.

This project provides practical experience in combining artificial intelligence with mobile application development.

Future improvements may include a more detailed disease information screen, additional testing with real-world leaf images, and improvements to model performance.

References

1. TensorFlow Documentation: https://www.tensorflow.org/
2. TensorFlow Lite Documentation: https://www.tensorflow.org/lite
3. TensorFlow Datasets: https://www.tensorflow.org/datasets
4. Android Developers: https://developer.android.com/
5. PlantVillage Dataset: https://www.tensorflow.org/datasets/catalog/plant_village
6. Project GitHub Repository: https://github.com/maira123669/LeafGuard.AI
7. Instructor Reference Repository: https://github.com/Uzair390-Del/NumberGuessingAppTfli